Linux Kernel Development, how it all works

Greg Kroah-Hartman

gregkh@linuxfoundation.org
git.sr.ht/~gregkh/kernel-development

CC BY NC SA



All of this is just my personal opinion, based on being a Linux kernel developer since 1999 and part of the Linux kernel security team since it was created in 2005.

Nothing in here reflects the opinion of the Linux Foundation or any other Linux kernel developer.

But hopefully I can convince them to agree with me.

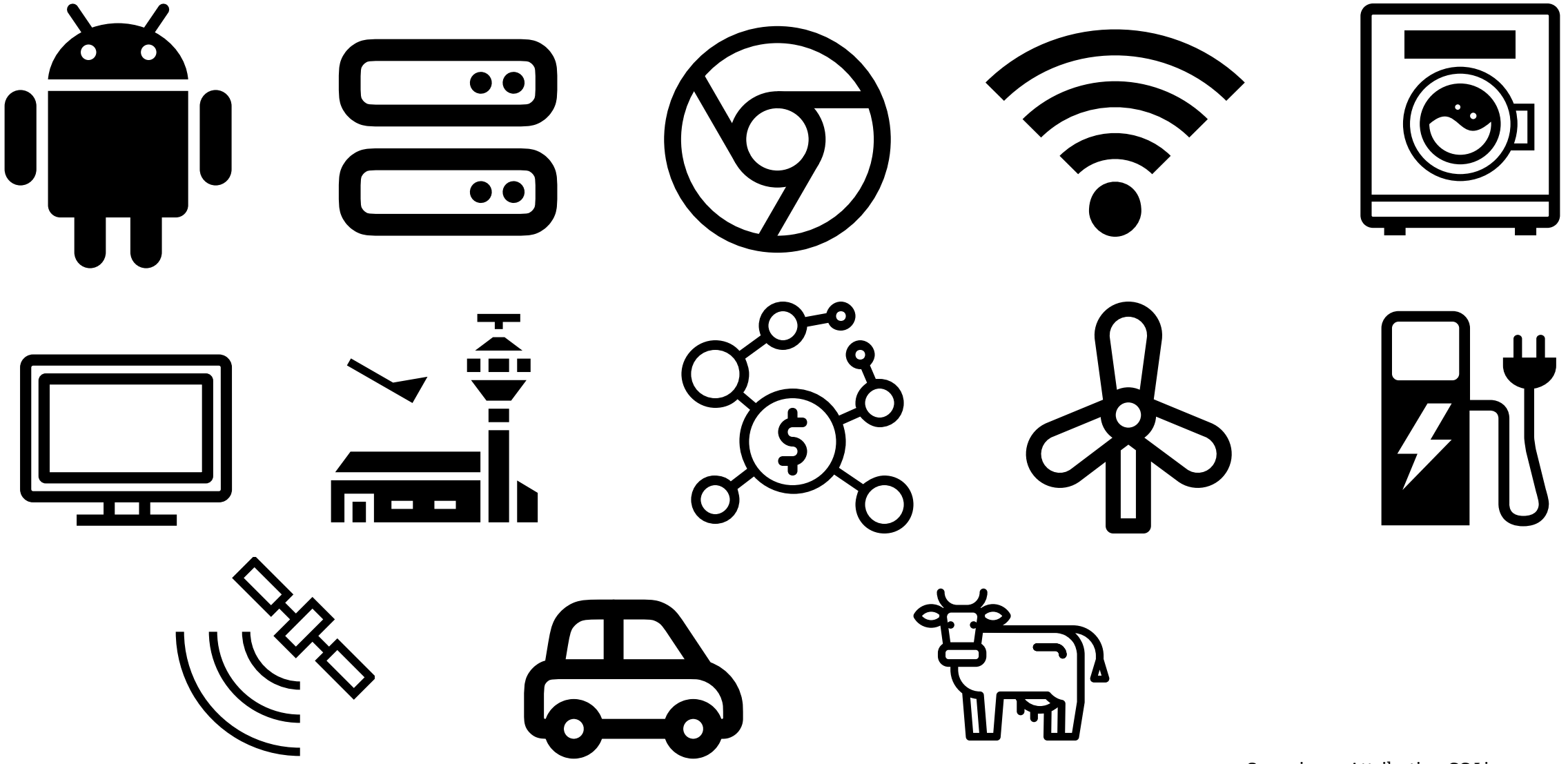
Linux Developer Community

5,221 developers

375+ employers

78,209 changes accepted

Linux runs the world



Some icons Attribution CC License, svgrepo.com

Linux size – overall

94,700 files

43,200,000 lines

Kernel release 7.2.0

Linux size – what you use

5%-10%

~10.9 changes per hour

Kernel release 7.2.0

New* release model

- Release every 2-3 months
- All releases are stable

* As of January, 2004

“Cambridge promise”

- We will not break userspace

– July 2007

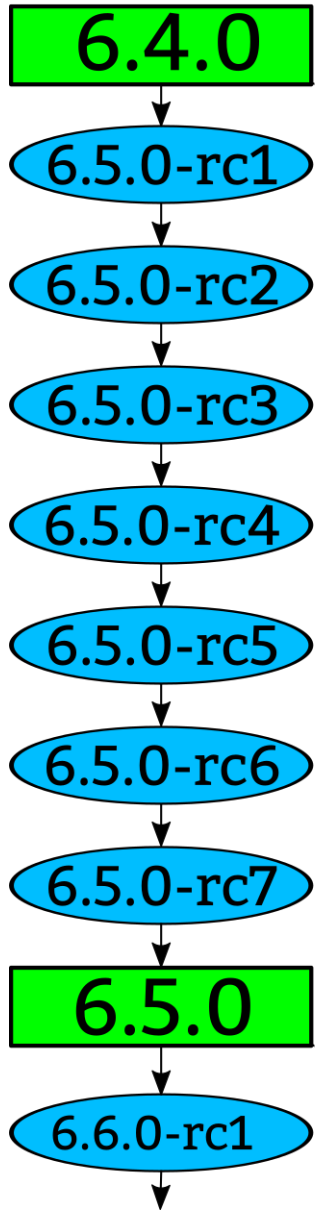
“Cambridge promise”

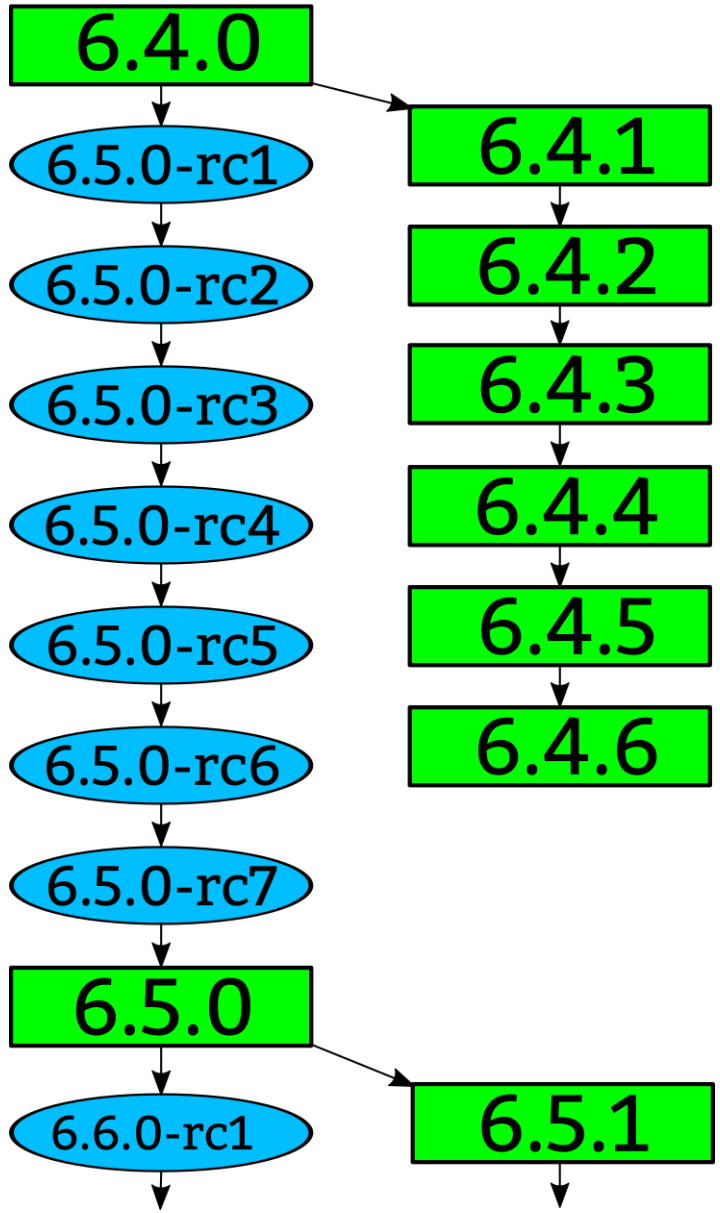
- We will not break userspace on purpose

– July 2007

Version numbers mean nothing

2.6.x → 3.x	2011
3.x → 4.x	2015
4.x → 5.x	2019
5.x → 6.x	2022
6.x → 7.x	2026


6.4.0
6.5.0-rc1
6.5.0-rc2
6.5.0-rc3
6.5.0-rc4
6.5.0-rc5
6.5.0-rc6
6.5.0-rc7
6.5.0
6.6.0-rc1


6.4.0
6.5.0-rc1
6.5.0-rc2
6.5.0-rc3
6.5.0-rc4
6.5.0-rc5
6.5.0-rc6
6.5.0-rc7
6.5.0
6.6.0-rc1
6.4.1
6.4.2
6.4.3
6.4.4
6.4.5
6.4.6
6.5.1

Stable kernel rules

- Bugfix
- Less than 100 lines
- New ids or quirks
- Must be in Linus's tree

https://www.kernel.org/doc/html/latest/process/stable-kernel-rules.html

Longterm kernels

- One picked per year
- Maintained for at least 2 years*

5.10 5.15 6.1 6.6 6.12 6.18

* sometimes longer

Longterm kernels

5.10	18 changes / day
5.15	21 changes / day
6.1	25 changes / day
6.6	29 changes / day
6.12	37 changes / day
6.18	55 changes / day

Kernel rate of change

5.10	18 changes / day
5.15	21 changes / day
6.1	25 changes / day
6.6	29 changes / day
6.12	37 changes / day
6.18	55 changes / day
7.2	100 changes / day

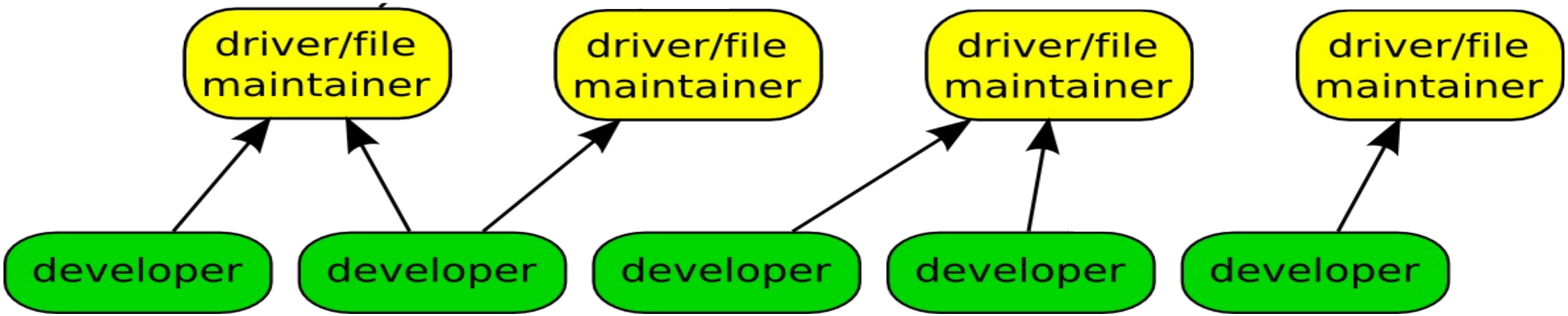
Kernel releases

- Every release is stable
- 18+ year old guarantee to not break things
- No fear to ever upgrade

More release information in greater detail:

http://www.kroah.com/log/blog/2018/02/05/linux-kernel-release-model/


developer
developer
developer
developer
developer


driver/file maintainer
driver/file maintainer
driver/file maintainer
driver/file maintainer
developer
developer
developer
developer
developer

```
commit ecf85e481a716cfe07406439fdc7ba9526bbfaeb
Author:     Robert Jarzmik <robert.jarzmik@free.fr>
AuthorDate: Tue Apr 21 20:33:10 2009 -0700
Commit:     Greg Kroah-Hartman <gregkh@suse.de>
CommitDate: Thu Apr 23 14:15:31 2009 -0700

    USB: otg: Fix bug on remove path without transceiver

    In the case where a gadget driver is removed while no
    transceiver was found at probe time, a bug in
    otg_put_transceiver() will trigger.

    Signed-off-by: Robert Jarzmik <robert.jarzmik@free.fr>
    Acked-by: David Brownell <dbrownell@users.sourceforge.net>
    Signed-off-by: Greg Kroah-Hartman <gregkh@suse.de>

--- a/drivers/usb/otg/otg.c
+++ b/drivers/usb/otg/otg.c
@@ -43,7 +43,8 @@ EXPORT_SYMBOL(otg_get_transceiver);
 void otg_put_transceiver(struct otg_transceiver *x)
 {
-       put_device(x->dev);
+       if (x)
+               put_device(x->dev);
 }
```

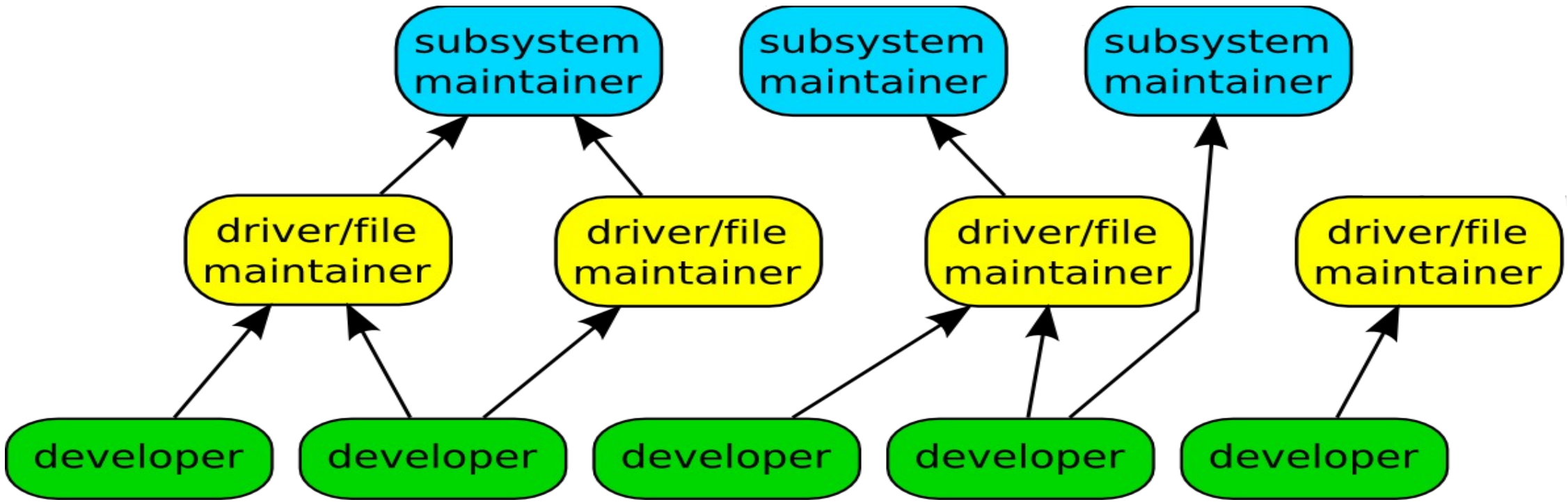
Developer's Certificate of Origin

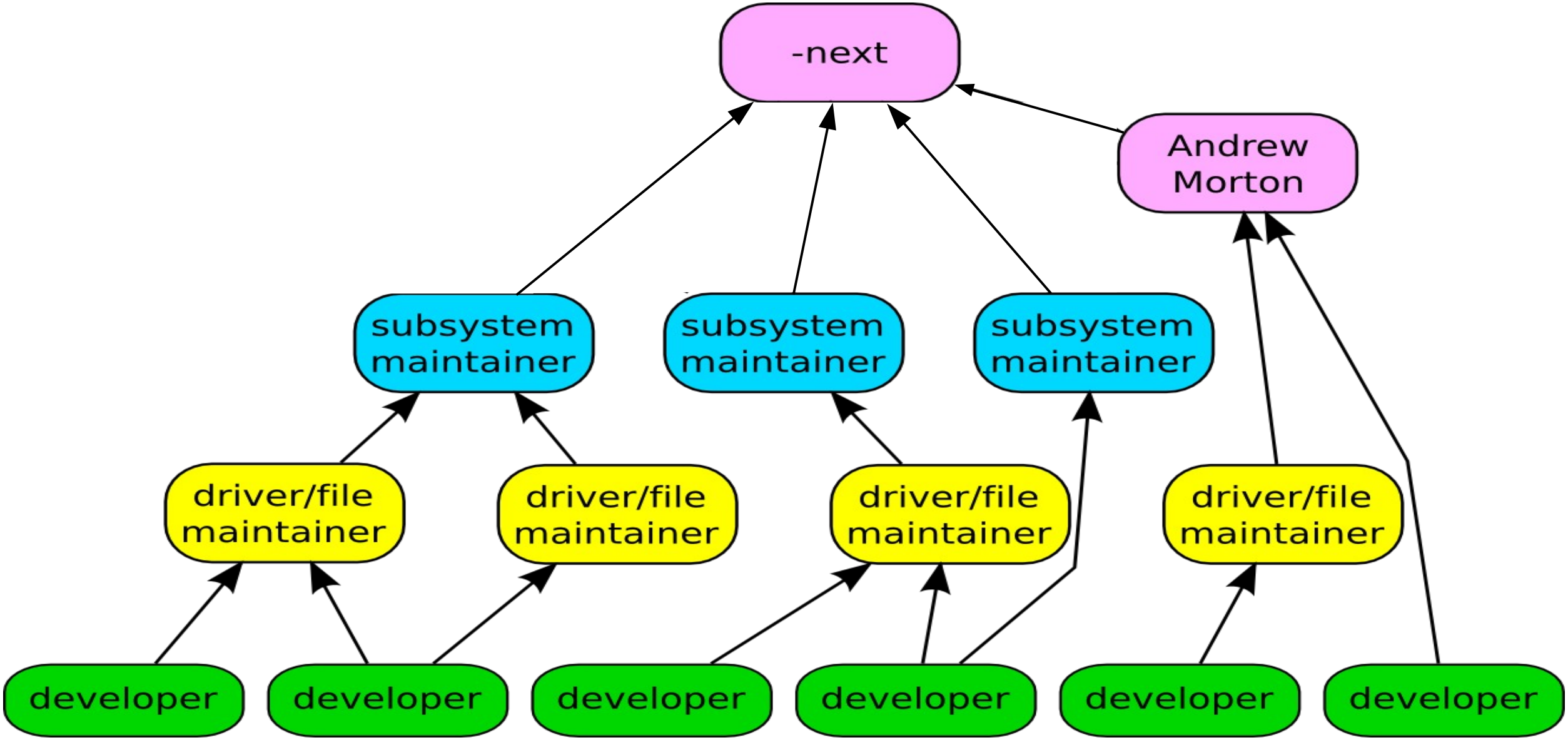
(a) I created this change; or

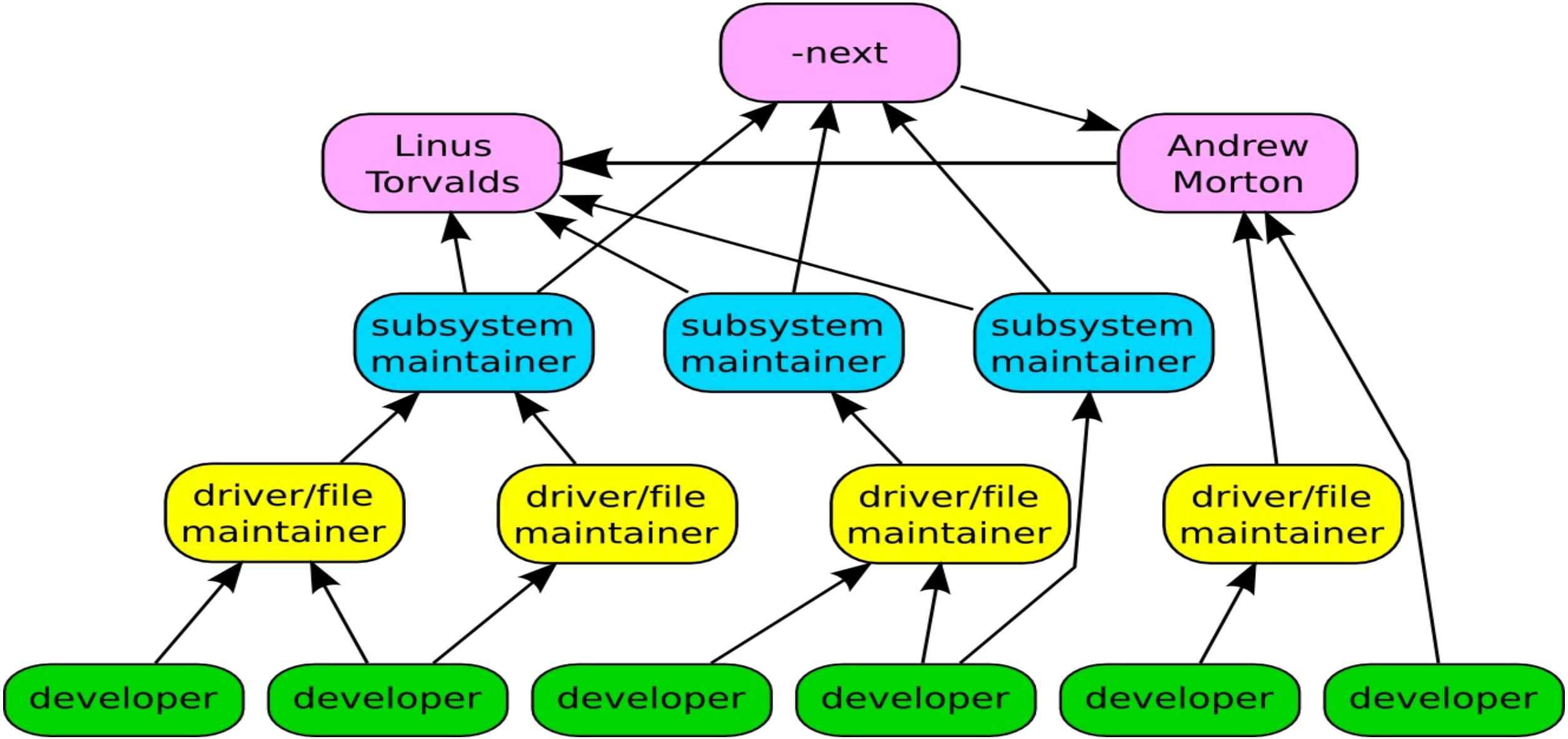
(b) Based this on a previous work with a compatible license; or

(c) Provided to me by (a), (b), or (c) and not modified

(d) This contribution is public.


subsystem maintainer
subsystem maintainer
subsystem maintainer
driver/file maintainer
driver/file maintainer
driver/file maintainer
driver/file maintainer
developer
developer
developer
developer
developer


-next
Andrew Morton
subsystem maintainer
subsystem maintainer
subsystem maintainer
driver/file maintainer
driver/file maintainer
driver/file maintainer
driver/file maintainer
developer
developer
developer
developer
developer
developer


-next
Linus Torvalds
Andrew Morton
subsystem maintainer
subsystem maintainer
subsystem maintainer
driver/file maintainer
driver/file maintainer
driver/file maintainer
driver/file maintainer
developer
developer
developer
developer
developer
developer

The world has changed

- 80%+ of the world's servers runs non-commercial distribution kernels*
- inter-company interactions achieve nothing
- The "community" does not sign NDAs

* Embedded it is like 99%, look at what is in your pocket or in your home

Kernel security team

Kernel security team

Reactive security, not proactive

Kernel security team

Other groups do proactive security

Kernel security team

- security@kernel.org
- Small group of kernel developers
- Do not represent any companies

Kernel security team

- Triage reports
- Drag in responsible developers
- Work to create a fix as soon as possible
- Get it merged into Linus's and stable trees

Kernel security team

- If you are brought in enough times, you are added to the alias to reduce the round-trip.

Kernel security team

- Fix the issue as soon as possible
- No embargoes longer than 7 days
- Do not do any kind of announcements

Kernel security policy

- Almost all bugs can be a “security” issue

Kernel security policy

- A fix for a known bug is better than the potential of a fix causing a future problem as future problems, when found, will be fixed then.

Kernel security policy

- We do NOT know your use case!

Kernel security policy

- We do NOT know your use case!
- We do NOT know what code you use!

Kernel security policy

- We do NOT know your use case!
- We do NOT know what code you use!
- We do NOT want to know any of this!

“It's hard to capture the fact that a bug can be super serious in one type of deployment, somewhat important in another, or no big deal at all -- and that the bug can be all of this at the same time. Vulnerability remediation is hard.”

– Ben Hawkes

https://blog.isosceles.com/what-is-a-good-linux-kernel-bug/

Kernel security policy

- Fix known bugs as soon as possible
- Get releases out to users quickly
- Does not work for hardware bugs*

* Hardware vendors think they are special,
They are not, they are just slow...

Hardware security issues

- Handled separately
- Encrypted restricted email list
- No NDAs
- Cross company / OS coordination
- Embargoes are tolerated*

* for now...

Hardware security issues

- How this works:

  https://www.kernel.org/doc/html/latest/process/embargoed-hardware-issues.html

Kernel security team

- Does not do any kind of announcements
- Can not assign CVEs*
- No early announcement list

* This is on purpose

No pre-disclosure at all!

- All “early notice” lists are leaks and should be considered public.

No pre-disclosure at all!

- All “early notice” lists are leaks and should be considered public.
- Unless your project is not used by anyone.

No pre-disclosure at all!

- All “early notice” lists are leaks and should be considered public.
- Unless your project is not used by anyone.
- Otherwise, why would your government allow it to exist?

Security fixes

- Happen at least once a week
- Look like any other bugfix
- Many not known to be security related until years later
- No differentiation between bug types
- No CVEs for anything

CVEs used to mean nothing for Linux

https://kernel-recipes.org/en/2019/talks/cves-are-dead-long-live-the-cve/

CVEs now mean something!

https://www.cve.org/Media/News/item/news/2024/02/13/kernel-org-Added-as-CNA

http://www.kroah.com/log/blog/2024/02/13/linux-is-a-cna/

kernel.org is now a CNA!

- Responsible for all kernel CVEs
- Community is in control
- CVEs assigned for all “vulnerabilities” fixed

Linux CVE resources

- **Contact us:**

  cve@kernel.org

- **Process documentation:**

  https://docs.kernel.org/process/cve.html

- **Public git repo:**

  https://git.kernel.org/pub/scm/linux/security/vulns.git/

- **List of all assigned CVEs:**

  https://lore.kernel.org/linux-cve-announce/

What is a vulnerability?

“An instance of one or more weaknesses in a Product that can be exploited, causing a negative impact to confidentiality, integrity, or availability; a set of conditions or behaviors that allows the violation of an explicit or implicit security policy.”

What is a kernel vulnerability?

- Any user-triggerable crash
- Memory use-after-free / leak / overflow
- Incorrect boundary checks
- Denial of service
- Logic errors
- Lots of other things

What is a not a kernel vulnerability?

- Data corruption / loss
- Performance issues
- Bugfixes that are not externally triggered

How are they assigned

- CVE team reviews every stable commit
- Votes if is, or is not, a vulnerability
- Each team member uses different methods
- Commits that are agreed get assigned
- Commits that are disagreed are discussed
- Review happens in public

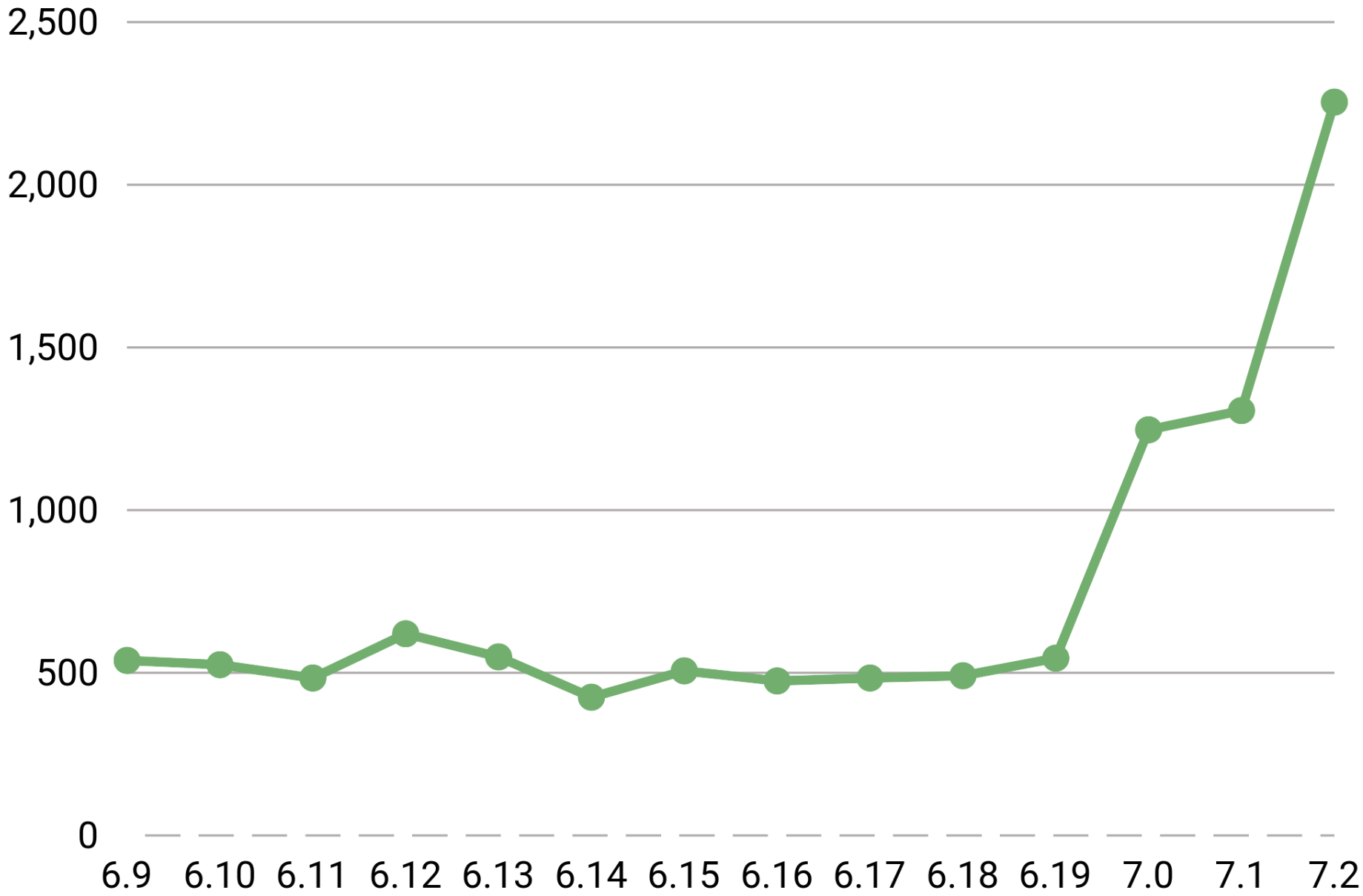
How are they assigned

- Community requests

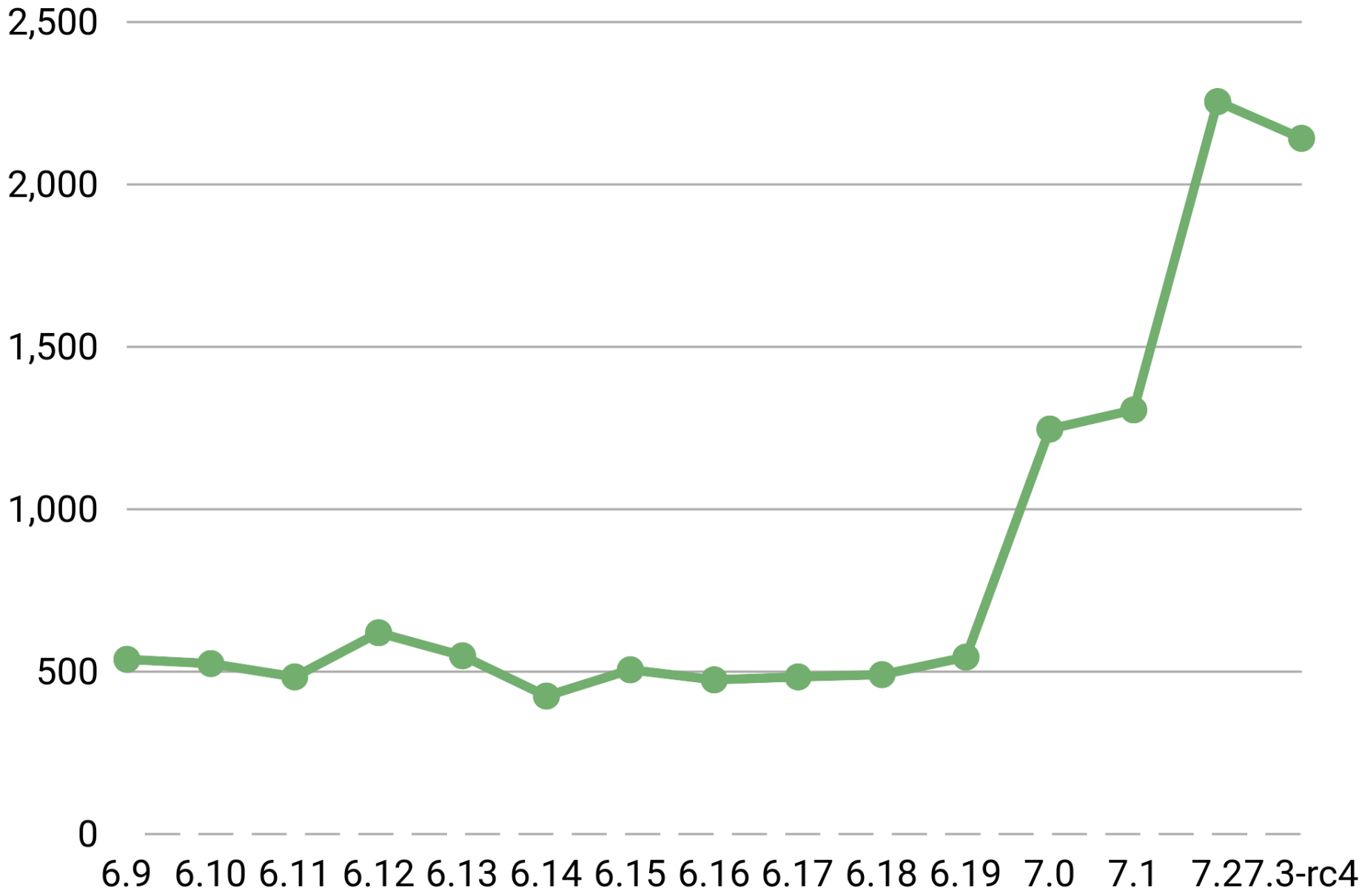
CVEs for everyone!

- Averaging 61 CVEs per day!
- Every CVE says what files are affected
- Every CVE says what versions are affected
- Very few CVEs are applicable for you!

CVE fixes per release



CVE fixes per release (with -rc)



Most CVEs are not for you!

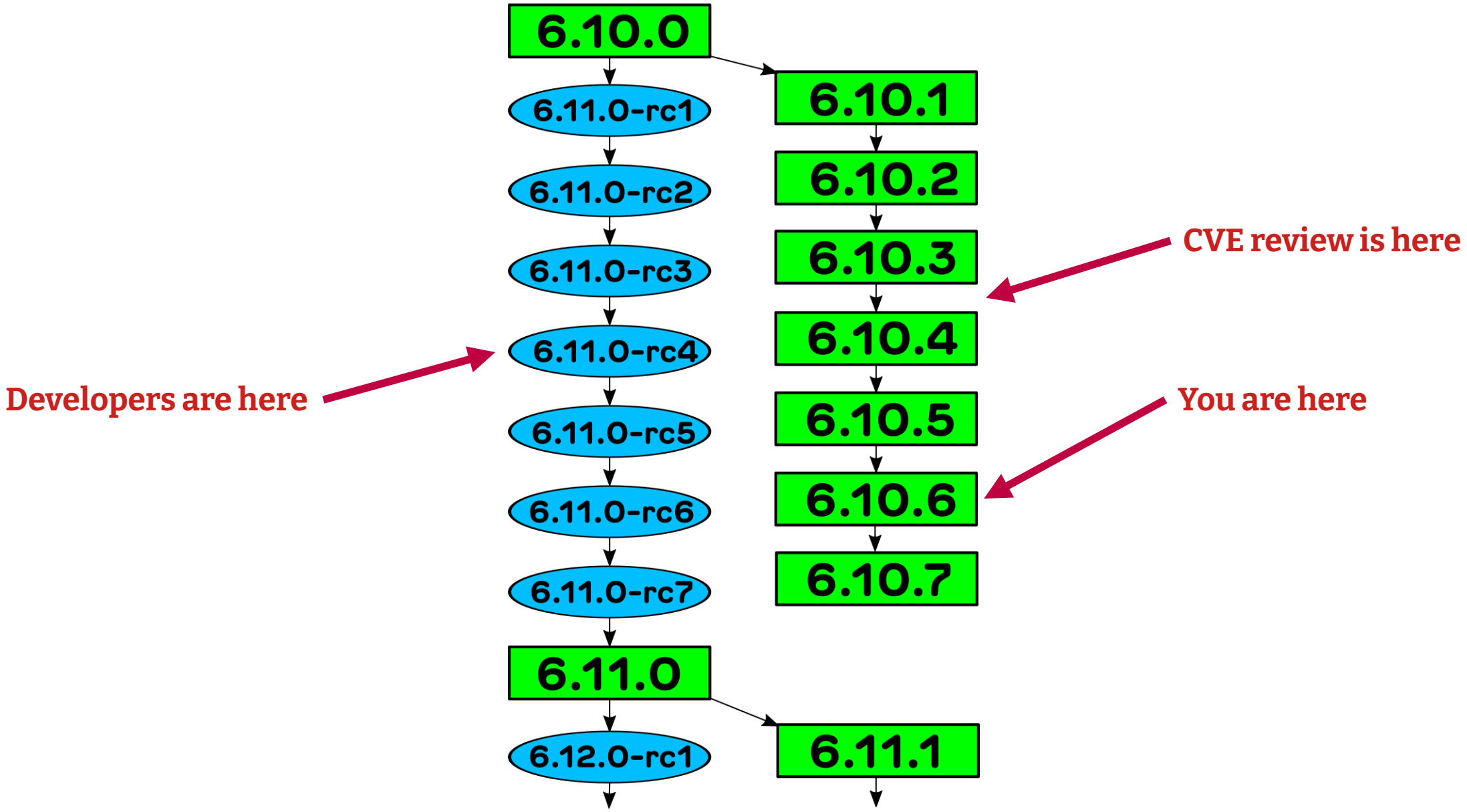
- Only you know your usecase!
- Only you know what files you use!

Some CVEs are for you!

- Real issues get fixed every week
- Ignoring them will affect your security

CVEs are assigned after-the-fact

- Usually 1-2 week delay
- Allows systems to be updated before public announcements
- CVEs only reference specific fix, not any previous changes needed.
- CVE fixes are NOT tested independently


6.10.0
6.11.0-rc1
6.11.0-rc2
6.11.0-rc3
6.11.0-rc4
6.11.0-rc5
6.11.0-rc6
6.11.0-rc7
6.11.0
6.12.0-rc1
6.10.1
6.10.2
6.10.3
6.10.4
6.10.5
6.10.6
6.10.7
6.11.1
Developers are here
CVE review is here
You are here

Staying secure

- Take all stable/longterm updates
- Community supported and tested as a whole
- Bonus is you get data corruption fixes and performance improvements for free!

“If you are not using the latest stable / longterm kernel, your system is insecure”

– me

Linux Kernel Development, how it all works

Greg Kroah-Hartman

gregkh@linuxfoundation.org
git.sr.ht/~gregkh/kernel-development

CC BY NC SA

Bonus Slides!

LLMs are now “good enough”

- Security bug reports / fixes are real
- All open source projects are flooded
- Everyone is reevaluating how to handle this

Network LLMs are “public”

- Any “found” bug will be seen by others.
- Network LLMs want to “please” you.
- Projects are treating them as public

“If AI can find vulnerabilities, it can fix them.
The only thing scarce now is collective will.”

– Jim Zemlin

“I’m condemned to use the tools of my enemy to defeat them.”

– Andor

How to fix a problem

```
llm --dangerously-skip-permissions \
    -p "Fix this code"             \
    --verbose                      \
  &> ~/tmp/fix.log
```

Use open models locally

- They are all “good enough”.
- Will find/fix almost all issues.
- Keeps everything private.
- Worry about the next model when you get it.
- Companies wasted $25 billion not doing this.

What you can do today

- Ignore the doom marketing

What you can do today

- Ignore the doom marketing
- Know what software you use/rely on

What you can do today

- Ignore the doom marketing
- Know what software you use/rely on
- Be able to update your systems on a continuous basis

What you can do today

- Ignore the doom marketing
- Know what software you use/rely on
- Be able to update your systems on a continuous basis
- Update your dependancy chains

What you can do today

- Run local models internally

What you can do today

- Run local models internally
- NEVER upload any non-public information

What you can do today

- Run local models internally
- NEVER upload any non-public information
- Fix the bugs you find today

What you can do today

- Run local models internally
- NEVER upload any non-public information
- Fix the bugs you find today
- Send the fix upstream to be merged
- Ignore the “model of tomorrow”

What you can do today

- Run local models internally
- NEVER upload any non-public information
- Fix the bugs you find today
- Send the fix upstream to be merged
- Ignore the “model of tomorrow”

What you can do today

- Securing Open Source in the Age of AI

  `https://openssf.org/resources/securing-open-source-in-the-age-of-ai-a-practical-guide/`

Whose side has more will?

It’s going to be a rocky 18 months…